

Guess Number Higher or Lower

An Intuitive Guide to Binary Search & Optimal Strategy

1. The Game Rules

Imagine a simple party game: I pick a secret number between **1** and **n** . Your goal is to figure out what that number is in as few guesses as possible.

Every time you make a guess, I give you immediate feedback using a pre-defined API method `guess(num)`:

- `-1`: My secret number is **lower** than your guess (`num > pick`). You guessed too high!
- `1`: My secret number is **higher** than your guess (`num < pick`). You guessed too low!
- `0`: Congratulations! Your guess is **correct** (`num == pick`).

2. Illustrative Example

Let's trace a sample round where $n = 10$ and the secret picked number is **6**:

Your Guess	API Result (<code>guess(num)</code>)	Meaning & Action
3	1	The secret number is higher than 3. Discard 1, 2, and 3!
8	-1	The secret number is lower than 8. Discard 8, 9, and 10!
6	0	Correct! You found the secret number.

API Contract Summary

The `guess(num)` function acts as an oracle. It tells you not just whether you're right or wrong, but *which direction* to head next.

Core Question: What is the absolute smartest strategy to guarantee finding the secret number in the minimum number of guesses?

3. Naive vs. Smart Strategy

The Naive Approach

Guess linearly: 1, 2, 3, 4, 5...

Worst case: n guesses.

If $n = 1,000,000$, you might need up to 1,000,000 questions! Extremely slow for large ranges.

The Intuitive Leap

Always guess the **midpoint** of the remaining valid range.

Since feedback tells us if the target is higher or lower, guessing the middle eliminates **50% of candidates** in a single turn!

4. Visual Walkthrough (Shrinking the Range)

Let $n = 16$ and secret picked number = **\$11\$**. See how picking the midpoint repeatedly halves the search space:

Round 1: Range [1..16] | Guess mid = 8

Target (11)

1 .. 7 (Discarded)

8 (Too Low!) .. 16 (Remaining)



Round 2: Range [9..16] | Guess mid = 12

9 .. 11 (Remaining)

12 (Too High!) .. 16 (Discarded)

Round 3: Range [9..11] | Guess mid = 10

9 (Low)

10 (Too Low!) .. 11

Round 4: Range [11..11] | Guess mid = 11

Found 11!

Why It Works: The Power of Order

This divide-and-conquer strategy is called **Binary Search**. It functions because numbers are naturally sorted. Every number strictly less than the target yields "too low", and every number strictly greater yields "too high"—creating a monotonic structure that guarantees we can discard 50% of candidate answers per guess.

5. Connecting Intuition to Python Code

Here is the complete Python solution implementing the midpoint elimination logic:

```
class Solution(object):
    def guessNumber(self, n):
        left, right = 1, n # 1. Initialize search range boundaries [1..n]
        while left <= right: # 2. Continue while valid search space remains
            mid = (left + right) // 2 # 3. Guess the midpoint of current range
            result = guess(mid) # 4. Query the API feedback oracle
            if result < 0: # 5. Too High! Discard mid and entire right half
                right = mid - 1
            elif result > 0: # 6. Too Low! Discard mid and entire left half
                left = mid + 1
            else: # 7. Correct! Found secret number
                return mid
```

6. Mapping Code to Number-Line Intuition

- `left, right`: Represents the active boundaries of our number line. Anything outside `[left, right]` has already been eliminated.
- `mid = (left + right) // 2`: Picks the exact center of our current search space to maximize the eliminated candidates regardless of the outcome.
- `right = mid - 1`: When the guess is too high, `mid` and all numbers greater than `mid` are removed from consideration.
- `left = mid + 1`: When the guess is too low, `mid` and all numbers smaller than `mid` are removed from consideration.

7. Why It's Fast – Complexity Comparison

Time Complexity: $\mathcal{O}(\log n)$ | **Space Complexity:** $\mathcal{O}(1)$

Because the candidate space is cut in half with every guess, the total number of turns grows logarithmically with n . Even for huge values of n , Binary Search finds the answer in just a few dozen checks!

Total Numbers (n)	Naive Linear Strategy ($\mathcal{O}(n)$)	Binary Search Strategy ($\mathcal{O}(\log_2 n)$)
100	Up to 100 guesses	At most 7 guesses
1,000,000 (1 Million)	Up to 1,000,000 guesses	At most 20 guesses
1,000,000,000 (1 Billion)	Up to 1,000,000,000 guesses	At most 30 guesses